

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Curso: Engenharia de Software

Projeto: Amigos do Fideles

Documento Técnico — Sprint 4: Arquitetura e Padrões de Projeto

Integrantes:

Arthur Henrique Silva Gomes

Daniel Alves Oliveira

Davi Quintanilha Fideles

Gabriel Bruno da Cruz

Henrique Fernandes Silva

1. Funcionalidades escolhidas

Para esta etapa de refatoração e aprimoramento arquitetural, o foco foi direcionado ao núcleo de processamento de reservas do sistema de trocas mútuas. As funcionalidades escolhidas para a aplicação dos padrões de projeto foram:

- **Cálculo Dinâmico de Tarifas:** A lógica que define o custo (em pontos ou valor de troca) das acomodações com base no tipo de quarto e no formato da reserva (individual, duplo, família).
- **Gerenciamento de Instâncias Globais:** O controle de acesso a recursos compartilhados do sistema, garantindo eficiência no consumo de memória.
- **Sistema de Notificação de Trocas:** O fluxo de eventos que ocorre imediatamente após a confirmação de um aluguel ou troca mútua entre os usuários.

2. Solução proposta: padrões e como foram utilizados

Para resolver os desafios de acoplamento e escalabilidade do código, foram implementados três padrões de projeto (Design Patterns) consagrados:

Strategy (Estratégia)

Utilizado no processo de precificação e cálculo de reservas (`AluguelService` e `TipoTarifa`). Em vez de manter métodos longos e complexos cheios de estruturas condicionais (`if/else` ou `switch`) para verificar o tipo de quarto, a lógica de cálculo foi isolada em classes de estratégia específicas. O contexto agora apenas delega o cálculo para a estratégia correspondente (ex.: `TarifaPadrao`, `TarifaFamilia`).

Observer (Observador)

Aplicado no fluxo de confirmação de reservas. Quando uma troca mútua é efetivada com sucesso, o sistema principal atua como o “sujeito” (Subject) e emite um evento. Outras partes do sistema (como módulos de envio de e-mail, atualização de saldo de pontos ou logs) atuam como “observadores” (Observers), escutando o evento e reagindo automaticamente sem travar o processo principal.

Singleton

Aplicado em componentes estruturais que exigem um ponto global de acesso único, conforme detalhado no tópico 4.

3. Justificativa da escolha

Justificativa para o Strategy

O sistema de trocas mútuas possui regras flexíveis que podem mudar futuramente (como a adição de novas categorias de acomodação ou promoções sazonais). O Strategy garante a conformidade com o Princípio Aberto/Fechado (Open/Closed Principle) do SOLID: o sistema está aberto para a inclusão de novas tarifas, mas fechado para modificações no serviço principal (AluguelService), evitando a quebra de código existente.

Justificativa para o Observer

Reduz drasticamente o acoplamento. A classe responsável por salvar a reserva no banco de dados não precisa conhecer os detalhes de como um e-mail de confirmação é montado e disparado. Isso permite que novos comportamentos pós-reserva sejam adicionados no futuro apenas criando novos observadores, mantendo as responsabilidades rigorosamente separadas.

4. Utilização do Singleton e justificativa da escolha

O padrão Singleton foi utilizado na classe de gerenciamento de configurações do sistema (ou no gerenciador de conexão com o banco de dados).

Justificativa: em uma plataforma web com múltiplas requisições simultâneas de usuários, instanciar repetidamente objetos que leem propriedades estáticas ou que mantêm pools de conexão gera um desperdício massivo de recursos e pode causar inconsistências de concorrência. O Singleton garante que exista apenas uma única instância desse gerenciador rodando na memória (JVM), fornecendo um ponto de acesso global seguro, sincronizado e com ciclo de vida controlado durante toda a execução da aplicação.

5. Benefícios obtidos com a nova arquitetura

A introdução destes padrões de projeto elevou a maturidade técnica do projeto “Amigos do Fideles”, trazendo os seguintes benefícios diretos:

- **Alta Coesão e Baixo Acoplamento:** As classes agora possuem responsabilidades únicas e bem definidas. O front-end (como os ajustes feitos no reserva.js) consegue se comunicar com uma API mais limpa e previsível.
- **Facilidade de Manutenção:** A detecção e correção de bugs tornou-se mais ágil. Um erro no cálculo de um quarto familiar, por exemplo, exige manutenção apenas na sua classe de estratégia específica, sem o risco de impactar o cálculo de quartos duplos.

- **Testabilidade:** A arquitetura modularizada facilitou a criação e execução de testes unitários (`AluguelServiceTest.java`), pois agora é possível “mockar” (simular) estratégias e observadores isoladamente.
- **Escalabilidade Segura:** O sistema está preparado para crescer de forma orgânica. A adição de novas funcionalidades não exigirá a reescrita da regra de negócio central, garantindo a sustentabilidade do código a longo prazo.